# Guia Completo de Skills nos 3 Ambientes do Claude

---

## 1. Claude.ai (site/app) — Skills via Interface

### Como instalar

Acesse **Settings** > **Customize** > **Skills**. Você pode ativar as skills da Anthropic que já vêm disponíveis, ou fazer upload de um arquivo ZIP com uma skill customizada. Para baixar skills da comunidade, acesse o diretório em `claude.com/connectors`.

Para usar, basta ir em **Customize** > **Skills** e usar o toggle para ligar ou desligar cada skill individualmente. Skills desativadas ficam completamente fora do contexto.

### Estrutura do ZIP para skills customizadas

Quando você cria uma skill própria para o claude.ai, o arquivo ZIP deve ter esta estrutura:

```
minha-skill.zip
└── skill/
    ├── SKILL.md        ← arquivo principal com frontmatter + instruções
    └── assets/         ← (opcional) arquivos de suporte, exemplos, templates
        └── exemplo.txt
```

O `SKILL.md` precisa ter o frontmatter no topo:

```
---
name: minha-skill
description: Descrição clara de quando usar esta skill. Seja específico.
---
# Instruções da skill aqui
```

### Pré-requisito importante

É necessário ter o **Code execution** habilitado em Settings > Capabilities. Skills estão disponíveis para planos Free, Pro, Max, Team e Enterprise.

### Como funciona por trás

Apenas o **frontmatter YAML** (nome e descrição) de cada skill é carregado no system prompt no início. O conteúdo completo da skill só é lido quando o Claude decide que ela é relevante para sua tarefa — isso é chamado de **progressive disclosure**.

### Custo de tokens

Cada skill usa apenas ~100 tokens durante o scan inicial de metadados. Quando ativada para uma tarefa específica, o conteúdo completo carrega com menos de 5k tokens. Arquivos de suporte dentro da skill só carregam se necessários.

### Você pode desligar?

Sim. Basta usar o toggle ao lado de cada skill para desabilitar. Skills desativadas não ficam disponíveis para o Claude.

---

## 2. Claude Code (CLI) — Skills via Arquivos

### Estrutura de arquivos

```
seu-projeto/
├── CLAUDE.md                  ← memória do projeto (mais importante)
└── .claude/
    ├── settings.json          ← configurações e permissões
    ├── rules/                 ← instruções modulares por caminho
    │   └── api-rules.md
    └── skills/                ← skills invocáveis com /nome-da-skill
        └── minha-skill/
            ├── SKILL.md       ← arquivo principal da skill
            └── templates/     ← arquivos de suporte (carregam sob demanda)

~/.claude/                     ← nível global (todos os projetos)
├── CLAUDE.md
└── skills/
    └── minha-skill/
        └── SKILL.md
```

**Skills globais** (`~/.claude/skills/`) ficam disponíveis em qualquer projeto. **Skills de projeto** (`.claude/skills/`) só existem naquele repositório. Quando há conflito de nome, a skill do projeto tem prioridade.

## Como criar uma skill

Crie um diretório em `~/.claude/skills/nome-da-skill/` e dentro dele um arquivo `SKILL.md` com frontmatter YAML no topo:

```
---
name: explain-code
description: Explains code with visual diagrams. Use when explaining how code
---
# Instruções da skill aqui
```

O campo `name` vira o comando `/explain-code`, e o `description` é o que o Claude usa para decidir quando carregar automaticamente.

## Skills com parâmetros (arguments)

Skills podem declarar argumentos nomeados no frontmatter, que são passados quando você invoca com `/nome arg1 arg2`:

```
---
name: review-pr
description: Reviews a pull request. Use when user wants feedback on a PR.
arguments:

  - name: pr_number
    description: The PR number to review
    required: true

  - name: focus
    description: Specific area to focus on (e.g., security, performance)
    required: false
---
# Revise o PR #{{pr_number}}
{{#if focus}}Foque especialmente em: {{focus}}{{/if}}
```

## Como invocar

Você pode usar de duas formas:

- **Automático**: o Claude lê o `description` e carrega a skill quando julgar relevante
- **Manual**: você digita `/nome-da-skill` diretamente no chat, com ou sem argumentos

## Boas práticas para escrever descriptions eficazes

A `description` é o mecanismo de ativação automática — ela precisa ser precisa. Escreva como se fosse instruir outra pessoa sobre *quando* usar a skill:

| Fraco | Forte |
|---|---|
| `Helps with code` | `Use when user asks to explain, document, or diagram how existing code works` |
| `For pull requests` | `Use when reviewing a PR, creating a PR description, or evaluating code changes` |
| `Writing tool` | `Use when user asks to write, edit, or improve documentation, READMEs, or changelogs` |

Inclua exemplos de frases que o usuário pode dizer para acionar a skill: `"Use when user says 'how does this work', 'explain this', or 'draw a diagram'"`.

## CLAUDE.md vs Rules vs Skills — qual usar?

| | CLAUDE.md | `.claude/rules/` | Skills |
|---|---|---|---|
| **Quando usar** | Padrões gerais do projeto, arquitetura, convenções que *sempre* se aplicam | Instruções específicas por tipo de arquivo ou caminho (`src/api/**`) | Workflows reutilizáveis que só fazem sentido em tarefas específicas |
| **Quando carrega** | Sempre, desde o início da sessão | Só quando o arquivo ativo bate o padrão de caminho | Sob demanda (automático por description ou manual por `/comando`) |
| **Exemplo** | "Use TypeScript strict mode, commits em inglês" | "Em arquivos de rota, sempre validar | `/review-pr`, `/explain-code`, `/write-changelog` |

| | | | |
|---|---|---|---|
| | | com Zod antes do handler" | |

### Custo de tokens e controle

Os arquivos de memória são lidos no início de cada sessão e consomem espaço no context window — por isso é importante mantê-los enxutos. Para coisas que você só precisa às vezes, coloque em `docs/` e referencie com `@docs/arquivo.md` ao invés de deixar no CLAUDE.md fixo.

> **Dica:** use `@` para referenciar um arquivo pontualmente numa conversa sem que ele fique sempre no contexto. Use skills para workflows que se repetem. Use CLAUDE.md só para o que é universal no projeto.

Mantenha o CLAUDE.md raiz pequeno e estável. Qualquer coisa específica de um tópico ou só relevante em alguns momentos é melhor em arquivos de rules modulares.

### Você pode desligar skills específicas?

Sim. Você pode configurar `claudeMdExcludes` nas settings para excluir arquivos específicos. Managed policy skills (de organização) não podem ser excluídas.

---

## 3. VSCode via Extensão (Cline, Continue, Cursor, etc.)

Aqui não existe o conceito nativo de skills — tudo passa pela **API**, e você configura manualmente. O equivalente é o **system prompt** da extensão.

### Como "instalar" skills

Na maioria das extensões:

- **Cline / Roo Code**: existe um campo de **Custom Instructions** nas configurações — cole o conteúdo da sua skill ali
- **Continue**: use o campo `systemMessage` no `config.json`
- **Cursor**: tem um campo de **Rules for AI** nas configurações do projeto (`.cursorrules`) e globais
- **Todas**: algumas extensões também leem um arquivo `CLAUDE.md` ou `AGENTS.md` na raiz do projeto automaticamente

### Como funciona por trás

Na API, você usa o campo `system` de cada requisição. Não há leitura automática — você é responsável por incluir o conteúdo relevante. As skills precisam ser injetadas como parte do system prompt a cada chamada.

Não existe progressive disclosure: todo o conteúdo que você coloca no system prompt vai para a API em **todas as requisições**.

### Custo de tokens

Tudo que está no system prompt é cobrado **em toda requisição**, sem progressive disclosure. Por isso na API/extensões o ideal é incluir só o que é realmente necessário para aquele contexto de uso. Não tente replicar 20 skills — escolha as 2-3 mais importantes e coloque apenas o essencial de cada uma.

### Alternativa: injeção condicional via hooks

Extensões mais avançadas como Cline permitem criar **hooks** que injetam contexto dinamicamente com base no tipo de arquivo ou comando. Isso imita o comportamento de progressive disclosure do Claude Code, mas exige configuração manual.

---

## 4. Skills 2.0 — A Geração Atual

Skills 2.0 é a reformulação lançada pela Anthropic em março de 2026. A mudança central é arquitetural: skills deixam de ser simples modelos de instruções e passam a ter um **feedback loop de auto-otimização** — com evals, A/B testing e injeção dinâmica de contexto.

### O que mudou em relação à versão anterior

| Aspecto | Skills 1.0 | Skills 2.0 |
|---|---|---|
| Arquitetura | Template estático de instruções | Feedback loop com auto-otimização |
| Comandos vs Skills | Separados | Unificados (`.claude/commands/` e `.claude/skills/` produzem o mesmo `/comando`) |

| | | |
|---|---|---|
| Contexto | Único por sessão | Subagentes isolados com contextos próprios (`context: fork`) |
| Dados externos | Estáticos (você escreve no SKILL.md) | Shell commands injetam dados ao vivo antes do processamento |
| Avaliação | Nenhuma estrutura formal | Sistema de evals com score (ex: 7/9 prompts passaram) |
| A/B Testing | Não disponível | Comparação paralela de versões integrada |
| Otimização | Manual | Otimização automática de descriptions |
| Controle de ferramentas | Limitado | `allowed-tools` restringe quais ferramentas a skill pode usar |
| Override de modelo | Não disponível | Cada skill pode declarar qual modelo usar |
| Hooks | Nenhum | Hooks de ciclo de vida (pré/pós execução) |

---

## Skill Creator Tool

O centerpiece do Skills 2.0 é uma ferramenta de criação que funciona em linguagem natural — você descreve o que quer, e ela gera, testa e otimiza a skill sem necessidade de código. Opera em 4 modos:

1. **Criar**: descreve o objetivo, ela monta o SKILL.md
2. **Testar**: define exemplos de entrada e saída esperada
3. **Otimizar**: reescreve a `description` para melhorar o acionamento automático
4. **Comparar**: roda A/B testing entre versões

---

## Sistema de Evals (testes estruturados)

Skills 2.0 trata skills como código: você escreve testes para elas.

```
---
name: summarize-pr
```

```
description: Summarizes pull request changes concisely.
evals:

  - prompt: "Summarize this PR: adds null check to user login"
    expected: Contains "null check" and "login" in under 3 sentences

  - prompt: "Summarize this PR: refactors database connection pool"
    expected: Mentions "refactor" and "database" without implementation detai
---
```

O sistema executa todos os prompts e retorna um score (`7/9 passaram`). É como um teste unitário para comportamento de AI — você sabe objetivamente se a skill está funcionando ou regredindo.

---

## A/B Testing integrado

Compara duas versões de uma skill rodando os mesmos inputs em paralelo:

```
/skill-creator compare summarize-pr summarize-pr-v2
```

O resultado mostra qual versão produziu outputs melhores nos seus casos de teste, com insights sobre o porquê. Permite aposentar versões antigas com confiança.

---

## Injeção dinâmica de contexto

Um dos recursos mais poderosos do 2.0. Shell commands no corpo da skill executam **antes** de Claude processar — a saída substitui o placeholder. Claude recebe dados reais, não comandos.

```
---
name: pr-review
description: Reviews the current open PR with live context.
context: fork
allowed-tools: Bash(gh *)
---

## Contexto atual do PR
Diff: !`gh pr diff`
Comentários: !`gh pr view --comments`
Status dos checks: !`gh pr checks`

## Tarefa
```

```
Com base no contexto acima, revise o PR identificando:

- Problemas de lógica
- Riscos de segurança
- Oportunidades de melhoria
```

Quando você invoca `/pr-review`, os três comandos `gh` executam primeiro, e Claude recebe os resultados reais no lugar dos placeholders.

---

## Controle granular por skill

O frontmatter do Skills 2.0 ganhou vários campos novos para controle fino de comportamento:

```yaml
---
name: deploy-check
description: Verifica pré-requisitos antes de um deploy.

# Restringe quais ferramentas Claude pode usar nessa skill
allowed-tools: Bash(git *), Bash(gh *), Read

# Claude não invoca automaticamente — só você com /deploy-check
disable-model-invocation: true

# Ou o inverso: só Claude invoca, você nunca vê o /comando
user-invocable: false

# Override do modelo para essa skill específica
model: claude-opus-4-6

# Roda em subagente isolado (não polui o contexto principal)
context: fork
---
```

| Campo | O que faz |
|---|---|
| `allowed-tools` | Lista branca de ferramentas disponíveis nessa skill |
| `disable-model-invocation` | Impede ativação automática — só manual |
| `user-invocable: false` | Impede invocação manual — só automática (background knowledge) |

| | |
|---|---|
| `model` | Define o modelo usado especificamente nessa skill |
| `context: fork` | Executa em subagente com contexto próprio isolado |

---

## Dois tipos fundamentais de skills

### Capability Uplift Skills

Elevam o teto de qualidade em áreas onde o Claude é naturalmente mais fraco. Ensinam *como fazer algo melhor*.

```
---
name: pixel-perfect-ui
description: Use when designing UI components that require precise visual spe
             color theory application, or layout mathematics.
---
# Diretrizes para UI pixel-perfect

## Grid e espaçamento
Sempre use múltiplos de 8px. Margens internas: 16px, 24px ou 32px...

## Teoria de cores
Para paletas, calcule contraste WCAG 2.1...
[centenas de linhas de referência que carregam sob demanda]
```

### Workflow / Preference Skills (Encoded Preferences)

Não ensinam "como fazer algo" — documentam *onde encontrar informações e em que ordem processá-las* para um workflow específico.

```
---
name: onboarding-review
description: Use when reviewing code from new engineers or when asked to eval
             a first PR from someone on the team.
---
# Review de onboarding

## Contexto

- Checklist de padrões: !`cat docs/engineering-standards.md`
- PRs anteriores do autor: !`gh pr list --author $AUTHOR --limit 5`

## Abordagem
```

```
1. Priorize feedback educativo sobre nitpicks
2. Explique o porquê de cada sugestão
3. Indique onde no codebase encontrar exemplos do padrão correto
```

---

## Estrutura completa de uma skill 2.0

Uma skill pode ter múltiplos arquivos além do `SKILL.md`:

```
minha-skill/
├── SKILL.md              ← Required: frontmatter + instruções principais
├── template.md           ← Opcional: template que Claude preenche
├── examples/
│   ├── good-output.md    ← Opcional: exemplos de saída esperada
│   └── bad-output.md     ← Opcional: exemplos do que evitar
└── scripts/
    └── validate.sh       ← Opcional: script executável via injeção dinâmica
```

Arquivos extras **não consomem contexto** até serem acessados — são lidos via Bash quando necessário.

---

## Os três níveis de carregamento

| Nível | Quando carrega | Custo de tokens | O que contém |
|---|---|---|---|
| **Metadados** | Sempre (startup) | ~100 tokens | Nome + description do frontmatter |
| **Instruções** | Quando a skill é ativada | < 5k tokens | Corpo completo do SKILL.md |
| **Recursos** | Conforme necessário | Sem limite prático | Arquivos extras lidos via Bash |

O nível 3 é a grande virada: você pode incluir documentação massiva, datasets, muitos exemplos — sem penalidade de contexto para o que não for usado.

---

## Pre-built Agent Skills da Anthropic

A Anthropic disponibiliza skills prontas para formatos de documento que podem ser usadas diretamente:

| Skill | O que faz |
|---|---|
| `pdf` | Extrai texto, tabelas e estrutura de PDFs |
| `docx` (Word) | Lê e gera documentos Word mantendo formatação |
| `xlsx` (Excel) | Lê planilhas, faz cálculos, gera novas planilhas |
| `pptx` (PowerPoint) | Lê e cria apresentações |

Disponíveis no claude.ai (toggle na UI), Claude Code (bundled) e API (via `agent_skills` na requisição).

---

### Disponibilidade por superfície

| Superfície | Skills 2.0 disponível? | Como acessar |
|---|---|---|
| Claude.ai | ✅ Sim | Settings > Skills + toggle |
| Claude Code (CLI) | ✅ Sim | `.claude/skills/` ou `~/.claude/skills/` |
| Claude API | ✅ Sim | Campo `agent_skills` na requisição |
| Claude Agent SDK | ✅ Sim | TypeScript e Python |
| VSCode (extensões) | ❌ Não nativo | Apenas via system prompt manual |

> **Importante:** skills não sincronizam entre superfícies. Uma skill criada no Claude Code não aparece automaticamente no claude.ai.

---

## 5. Comparativo Final

| | claude.ai | Claude Code (CLI) | VSCode (extensão/API) |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Onde ficam** | Upload via UI (ZIP) | Arquivos no filesystem | System prompt da extensão |
| **Ativação** | Toggle na UI | Automática ou `/comando` | Manual no system prompt |
| **Progressive disclosure** | ✅ Sim | ✅ Sim | ❌ Não |
| **Custo de tokens** | Só quando usada | Frontmatter sempre + conteúdo sob demanda | Tudo cobrado sempre |
| **Suporte a arguments** | ❌ Não | ✅ Via frontmatter | ❌ Não (manual) |
| **Skills globais vs projeto** | ❌ Por conta do usuário | ✅ `~/.claude/` vs `.claude/` | ❌ Não há distinção nativa |
| **Injeção dinâmica (!comando)** | ❌ Não | ✅ Sim (Skills 2.0) | ❌ Não |
| **Evals e A/B Testing** | ❌ Não | ✅ Sim (Skills 2.0) | ❌ Não |
| **context: fork** | ❌ Não | ✅ Sim (Skills 2.0) | ❌ Não |
| **allowed-tools por skill** | ❌ Não | ✅ Sim (Skills 2.0) | ❌ Não |
| **Override de modelo** | ❌ Não | ✅ Sim (Skills 2.0) | ❌ Não |
| **Você pode desligar?** | ✅ Toggle individual | ✅ Via excludes ou não criando | ✅ Removendo do system prompt |
| **Skills da Anthropic** | ✅ Disponíveis prontas | ✅ Bundled skills | ❌ Precisa criar manualmente |

A grande vantagem do claude.ai e do Claude Code sobre a API/extensões é exatamente essa arquitetura de **progressive disclosure** — você pode ter muitas

skills disponíveis sem pagar tokens por todas elas em cada conversa. Com Skills 2.0, o Claude Code ainda se destaca com evals, A/B testing e injeção dinâmica, tornando-o o ambiente mais poderoso para criar e otimizar skills.